

CPSC 330 Lecture 18: Introduction to deep learning and computer vision

Firas Moosvi

Announcements

- HW7 was due yesterday
- HW8 has been released (due next week Monday)
 - Almost there! Hang in there 😊
- Midterm 2 grading is in progress.

iClicker 18.0

iClicker cloud join link: <https://join.iclicker.com/YJHS>

Select all of the following statements which are TRUE for you.

- a. I found the multiple-choice questions challenging.
- b. The coding questions took a lot of time.
- c. I didn't like the format of the midterm.
- d. I appreciated the mix of coding, conceptual, and multiple-choice questions.
- e. I felt the midterm was a good reflection of what we cover in the lectures and homework assignments.

iClicker 18.1

iClicker cloud join link: <https://join.iclicker.com/YJHS>

Select all of the following statements which are TRUE.

- a. It's possible to use word2vec embedding representations for text classification instead of bag-of-words representation.
- b. The topic model approach we used in the last lecture, Latent Dirichlet Allocation (LDA), is an unsupervised approach.
- c. In an LDA topic model, the same word can be associated with two different topics with high probability.
- d. In an LDA topic model, a document is a mixture of multiple topics.
- e. If I train a topic model on a large collection of news articles with $K = 10$, I would get 10 topic labels (e.g., sports, culture, politics, finance) as output.

Multiclass classification

- So far we have been talking about binary classification
- Can we use these classifiers when there are more than two classes?
 - “ImageNet” computer vision competition, for example, has 1000 classes
- Can we use decision trees or KNNs for multi-class classification?
- What about logistic regression?

Multinomial logistic regression

Softmax Function for Probabilities

Given an input, the probability that it belongs to class $j \in \{1, 2, \dots, K\}$ is calculated using the **softmax function**:

$$P(y = j \mid x_i) = \frac{e^{w_j^\top x_i + b_j}}{\sum_{k=1}^K e^{w_k^\top x_i + b_k}}$$

- x_i is the i^{th} example
- w_j is the weight vector for class j .
- b_j is the bias term for class j .
- K is the total number of classes.

Making Predictions

1. Compute Probabilities:

For each class j , compute the probability $P(y = j \mid x_i)$ using the softmax function.

2. Select the Class with the Highest Probability:

The predicted class () is:

$$\hat{y} = \arg \max_{j \in \{1, \dots, K\}} P(y = j \mid x_i)$$

Binary vs multinomial logistic regression

Aspect	Binary Logistic Regression	Multinomial Logistic Regression
Target variable	2 classes (binary)	More than 2 classes (multi-class)
Getting probabilities	Sigmoid	Softmax
parameters	d weights, one per feature and the bias term	d weights and a bias term per class
Output	Single probability	Probability distribution over classes
Use case	Binary classification (e.g., spam detection)	Multi-class classification (e.g., image classification)

Image classification

Have you used search in Google Photos? You can search for “my photos of cat” and it will retrieve photos from your libraries containing cats. This can be done using **image classification**, which is treated as a supervised learning problem, where we define a set of target classes (objects to identify in images), and train a model to recognize them using labeled example photos.

Image classification

Image classification is not an easy problem because of the variations in the location of the object, lighting, background, camera angle, camera focus etc.

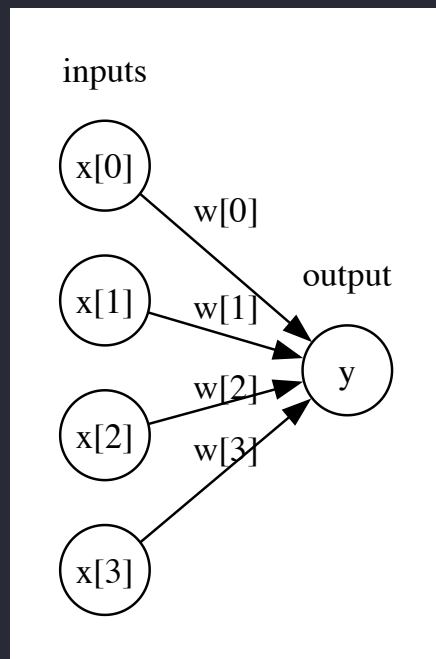


Neural networks

- Neural networks are perfect for these types of problems where local structures are important.
- A significant advancement in image classification was the application of **convolutional neural networks** (ConvNets or CNNs) to this problem.
 - ImageNet Classification with Deep Convolutional Neural Networks
 - Achieved a winning test error rate of 15.3%, compared to 26.2% achieved by the second-best entry in the ILSVRC-2012 competition.
- Let's go over the basics of a neural network.

Introduction to neural networks

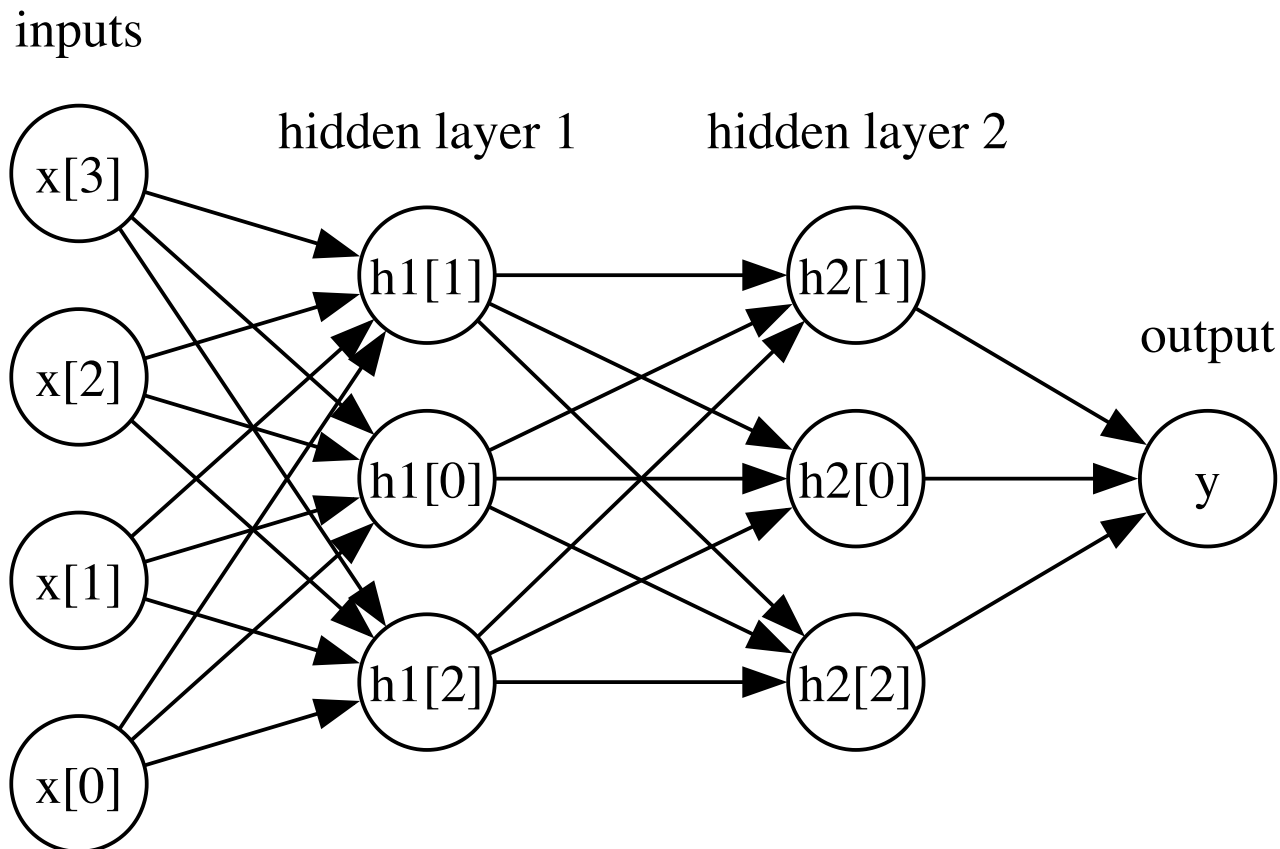
- Neural networks can be viewed a generalization of linear models where we apply a series of transformations.
- Here is graphical representation of a logistic regression model.
- We have 4 features: $x[0]$, $x[1]$, $x[2]$, $x[3]$



Adding a layer of transformations

One more layer of transformations

- Now we are adding one more layer of transformations.



Neural networks

- With a neural net, you specify the number of features after each transformation.
 - In the above, it goes from 4 to 3 to 3 to 1.
- To make them really powerful compared to the linear models, we apply a non-linear function to the weighted sum for each hidden node.
- Neural network = neural net
- Deep learning ~ using neural networks

Why neural networks?

- They can learn very complex functions.
 - The fundamental tradeoff is primarily controlled by the **number of layers** and **layer sizes**.
 - More layers / bigger layers → more complex model.
 - You can generally get a model that will not underfit.
- They work really well for structured data:
 - 1D sequence, e.g. timeseries, language
 - 2D image
 - 3D image or video
- They've had some incredible successes in the last 12 years.
- Transfer learning (coming later today) is really useful.

Why not neural networks?

- Often they require a lot of data.
- They require a lot of compute time, and, to be faster, specialized hardware called GPUs.
- They have huge numbers of hyperparameters
 - Think of each layer having hyperparameters, plus some overall hyperparameters.
 - Being slow compounds this problem.
- They are not interpretable.
- I don't recommend training them on your own without further training
- Good news
 - You don't have to train your models from scratch in order to use them.
 - I'll show you some ways to use neural networks without training them yourselves.

Deep learning software

The current big players are:

1. PyTorch
2. TensorFlow

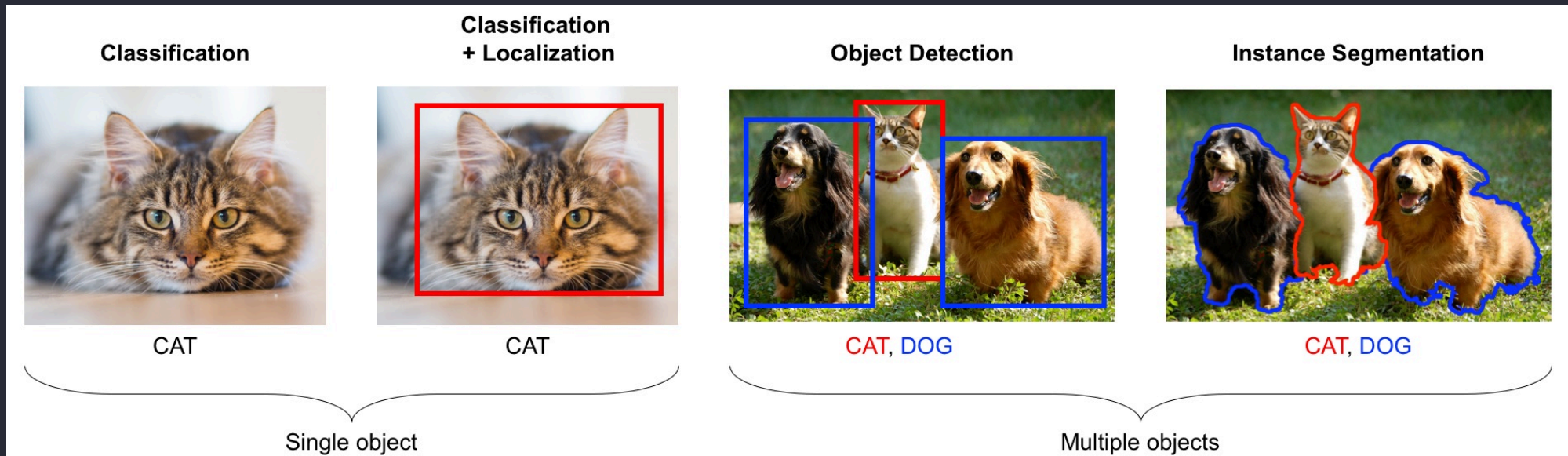
Both are heavily used in industry. If interested, see [comparison of deep learning software](#).

Introduction to computer vision

- **Computer vision** refers to understanding images/videos, usually using ML/AI.
- In the last decade this field has been dominated by deep learning. We will explore **image classification** and **object detection**.

Introduction to computer vision

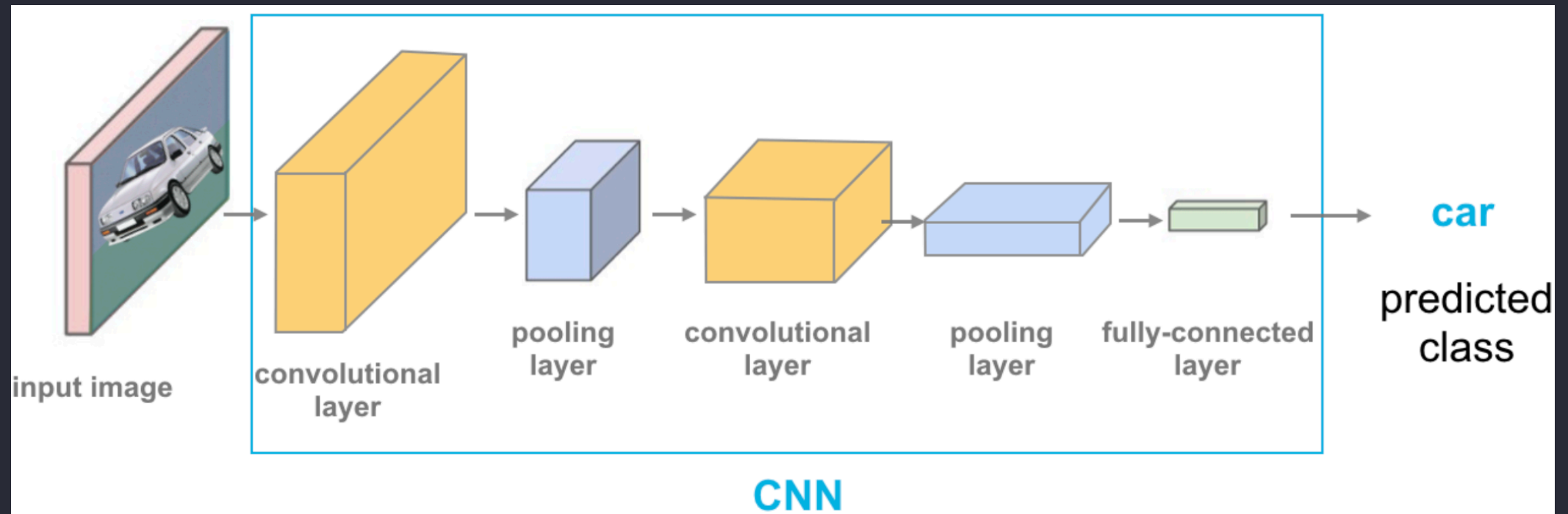
- image classification: is this a cat or a dog?
- object localization: where is the cat in this image?
- object detection: What are the various objects in the image?
- instance segmentation: What are the shapes of these various objects in the image?
- and much more...



Pre-trained models

- In practice, very few people train an entire CNN from scratch because it requires a large dataset, powerful computers, and a huge amount of human effort to train the model.
- Instead, a common practice is to download a pre-trained model and fine tune it for your task. This is called **transfer learning**.
- Transfer learning is one of the most common techniques used in the context of computer vision and natural language processing.
- It refers to using a model already trained on one task as a starting point for learning to perform another task.

Pre-trained models out-of-the-box



- Let's first apply one of these pre-trained models to our own problem right out of the box.

Pre-trained models out-of-the-box

- We can easily download famous models using the `torchvision.models` module. All models are available with pre-trained weights (based on ImageNet's 224 x 224 images)
- We used a pre-trained model vgg16 which is trained on the ImageNet data.
- We preprocess the given image.
- We get prediction from this pre-trained model on a given image along with prediction probabilities.
- For a given image, this model will spit out one of the 1000 classes from ImageNet.

Pre-trained models out-of-the-box

- Let's predict labels with associated probabilities for unseen images



Class	Probability score
tiger cat	0.353

Pre-trained models out-of-the-box

- We got these predictions without “doing the ML ourselves”.
- We are using **pre-trained vgg16** model which is available in `torchvision`.
 - `torchvision` has many such pre-trained models available that have been very successful across a wide range of tasks: AlexNet, VGG, ResNet, Inception, MobileNet, etc.
- Many of these models have been pre-trained on famous datasets like **ImageNet**.
- So if we use them out-of-the-box, they will give us one of the ImageNet classes as classification.

Pre-trained models out-of-the-box

- Let's try some images which are unlikely to be there in ImageNet.
- It's not doing very well here because ImageNet doesn't have proper classes for these images.

Pre-trained models out-of-the-box

- Here we are Pre-trained models out-of-the-box.
- Can we use pre-trained models for our own classification problem with our classes?
- Yes!! We have two options here:
 1. Add some extra layers to the pre-trained network to suit our particular task
 2. Pass training data through the network and save the output to use as features for training some other model

Pre-trained models to extract features

Pre-trained models to extract features

- Once we extract these feature vectors for all images in our training data, we can train a machine learning classifier such as logistic regression or random forest.
- This classifier will be trained on our classes using feature representations extracted from the pre-trained models.
- Let's try this out.
- It's better to train such models with GPU. Since our dataset is quite small, we won't have problems running it on a CPU.

Pre-trained models to extract features

Let's look at some sample images in the dataset.



Dataset statistics

Here is the stat of our toy dataset.

```
Classes: ['beet_salad', 'chocolate_cake', 'edamame', 'french_fries', 'pizza', 'spring_rolls', 'sushi']  
Class count: 40, 38, 40  
Samples: 283  
First sample: ('data/food/train/beet_salad/104294.jpg', 0)
```


Pre-trained models to extract features

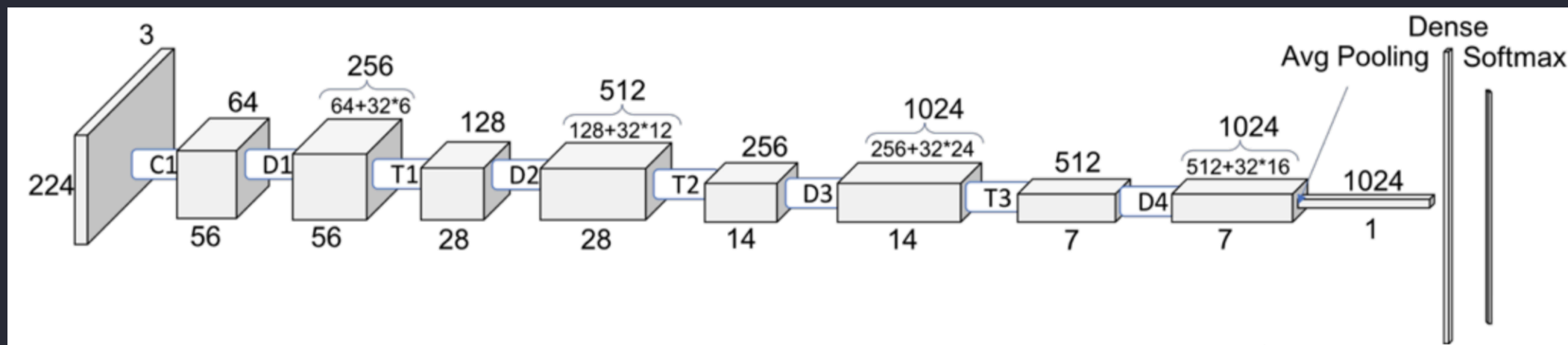
- Now for each image in our dataset, we'll extract a feature vector from a pre-trained model called densenet121, which is trained on the ImageNet dataset.

Shape of the feature vector

- Now we have extracted feature vectors for all examples. What's the shape of these features?

```
torch.Size([283, 1024])
```

- The size of each feature vector is 1024 because the size of the last layer in densenet architecture is 1024.



Source

A feature vector given by densenet

- Let's examine the feature vectors.

	0	1	2	3	4	5	
0	0.000594	0.005223	0.002873	0.001198	0.095008	0.586996	0.000
1	0.000409	0.002568	0.005813	0.000984	0.144840	0.398514	0.000
2	0.000416	0.001159	0.003767	0.003172	0.108386	0.586645	0.000
3	0.000497	0.005999	0.002178	0.002357	0.148310	0.246634	0.000
4	0.000736	0.004440	0.005242	0.002650	0.117545	0.587100	0.000

5 rows × 1024 columns

- The features are hard to interpret but they have some important information about the images which can be useful for classification.

Logistic regression with the extracted features

- Let's try out logistic regression on these extracted features.

Training score: 1.0

Validation score: 0.8208955223880597

- This is great accuracy for so little data and little effort!!!

Sample predictions

Let's examine some sample predictions on the validation set.

Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: sushi
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: beet_salad
True: beet_salad



Predicted: chocolate_cake
True: beet_salad



Object detection

- Another useful task and tool to know is object detection using YOLO model.
- Let's identify objects in a sample image using a pretrained model called YOLO8.
- List the objects present in this image.



Object detection using YOLO

Let's try this out using a pre-trained model.

```
1 from ultralytics import YOLO
2 model = YOLO("yolov8n.pt") # pretrained YOLOv8n model
3
4 yolo_input = "data/yolo_test/3356700488_183566145b.jpg"
5 yolo_result = "data/yolo_result.jpg"
6 # Run batched inference on a list of images
7 result = model(yolo_input) # return a list of Results objects
8 result[0].save(filename=yolo_result)
```

image 1/1 /Users/firasm/Sync/Teaching/2024_2025/2024W1/cpsc330/cpsc330-slides/website/data/yolo_test/3356700488_183566145b.jpg: 512x640 4 persons, 2 cars, 1 stop sign, 103.3ms
Speed: 2.5ms preprocess, 103.3ms inference, 5.7ms postprocess per image at shape (1, 3, 512, 640)

'data/yolo_result.jpg'

Object detection output



Summary

- Neural networks are a flexible class of models.
 - They are particularly powerful for structured input like images, videos, audio, etc.
 - They can be challenging to train and often require significant computational resources.
- The good news is we can use pre-trained neural networks.
 - This saves us a huge amount of time/cost/effort/resources.
 - We can use these pre-trained networks directly or use them as feature transformers.